SYDE 671 Assignment 3 Review

In this blog we will be reviewing the concepts from assignment 3. Assignment 3 focuses on image blending and stitiching multiple photographs to create one larger composite photograph.

Section 1: Homographies, Warping and Mosaicing

In this section we will look at the fundamentals of masaicing by manually picking the cordinates to warp a source image to a target image. I have chosen few different pairs of images to implement warping on. The first set is the classical chessboard example where we take an image with a chessboard on a 3D plane with background and warp to a target image of chessboard on a 2D plane. The second example is pictures I took in the park the other day and the third is some pictures I took inside the house. I will discuss why the pictures that I took in the park were not good images to implement image warping on and why the chessboard example works better.

Let's begin by first looking at the code and how we calculated the homographies and implemented warping.

```
import numpy as np
import cv2
import matplotlib.pyplot a

img1 = cv2.imread(r"C:\Use
img2 = cv2.imread(r"C:\Use

img1 = cv2.cvtColor(img1,
img2 = cv2.cvtColor(img2,

pts1 = np.array([[243, 65]
pts2 = np.array([[220, 108

def computeH(im1_pts, im2_
                  n = im1_
                  if n < 4


                  A = []
                  for i in
```

```
                                A = np.a

                                _, _, Vt
                                h = Vt[-1

                                H = h.re
                                if H[2, 2

                                return H

                            def visualize_correspondenc

                                fig, axes

                                axes[0].i
                                axes[0].s
                                axes[0].s
                                axes[0].a

                                axes[1].i
                                axes[1].s
                                axes[1].s
                                axes[1].a

                                plt.tight
                                plt.show

                            def get_bounding_box(im_sh
                                """
                                Predicts
                                to preve
                                """
                                h, w = im
                                corners =

                                ]).T  # S

                                transfor

                                z_vals =
                                z_vals[np
                                transfor

                                x_coords
                                y_coords

                                all_x = n
                                all_y = n

                                x_min = n
```

```
                                                x_max = r
                                                y_min = r
                                                y_max = r

                                                return x_

                                        def warpImageNearestNeighb
                                                """
                                                Warps ima
                                                """
                                                h, w = im
                                                channels

                                                x_min, x_
                                                out_w = x
                                                out_h = y

                                                x_indices
                                                y_indices
                                                X_out, Y_

                                                num_pixel
                                                ones = np
                                                coords_ou

                                                H_inv = r
                                                coords_sr
                                                coords_sr

                                                X_src = c
                                                Y_src = c

                                                X_nn = np
                                                Y_nn = np

                                                valid_mas

                                                X_nn_clam
                                                Y_nn_clam

                                                if channe
```

```
                                                else:
```

```
                                                alpha_mas
```

```
                                                    return im


                                        def warpImageBilinear(im,
                                                    """
                                                    Warps ima
                                                    """
                                                    h, w = im
                                                    channels

                                                    x_min, x_
                                                    out_w = x
                                                    out_h = y

                                                    x_indices
                                                    y_indices
                                                    X_out, Y_

                                                    num_pixel
                                                    ones = np
                                                    coords_ou

                                                    H_inv = n
                                                    coords_sr
                                                    coords_sr

                                                    X_src = c
                                                    Y_src = c

                                                    x0 = np.f
                                                    x1 = x0 +
                                                    y0 = np.f
                                                    y1 = y0 +

                                                    wa = (x1
                                                    wb = (X_s
                                                    wc = (x1
                                                    wd = (X_s

                                                    valid_mas

                                                    x0_c = np
                                                    x1_c = np
                                                    y0_c = np
                                                    y1_c = np

                                                    if channe
```

```
                else:

                    alpha_ma

                    return i

visualize_correspondences(

im_warped, alpha = warpIma
im_warped_nn, alpha_nn = w

fig, axes = plt.subplots(1

axes[0].imshow(im_warped_n
axes[0].set_title("Nearest
axes[0].axis('off')

axes[1].imshow(im_warped)
axes[1].set_title("Bilinea
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

I start by importing the images and converting them from the default BGR format to RGB format. After loading the images, I use pixpy to get the cordinates for the source and target images to create the homographies with and manually input them as arrays in my code.

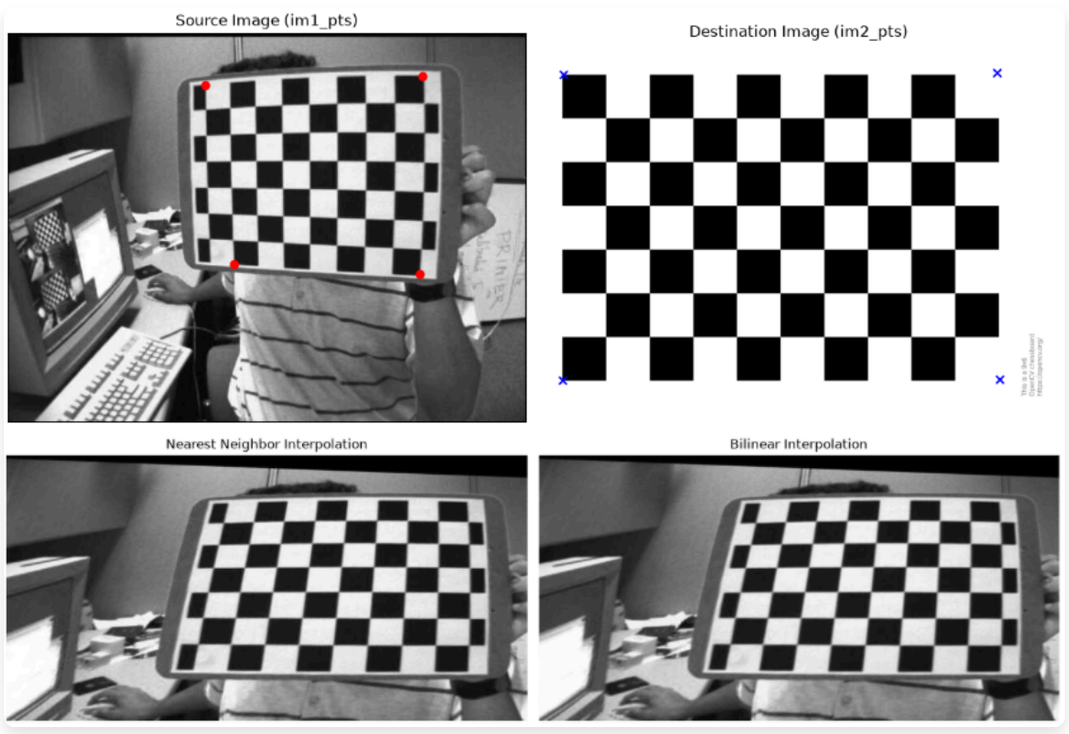
The computeH function then takes the arrays as input and calculates the homography matrix taking the cordinate arrays as the input. The first check the function does is to check for the number of cordinate pairs in the arrays, if there are less than four cordinate pairs in either of the arrays, it gives an error. The function then uses Direct Linear Transformation (DLT) to create matrix A of size 2N X 9. Where each point correspondence contributes to two rows of linear equations representing Ah = 0. Then the function then factorizes the matri using the Singular Value Decomposition (SVD) and identifies the optimal least-squares solution which forms the parameters of the homography matrix. Finally, the function reshapes the vector into a 3 x 3 matrix format and scales it by its bottom-right element such that magnitude of H equals 1, before returning the final homography matrix.

The get_bounding_box function then calculates the spatial boundaries of the warped image taking the source image dimensions and homography matrix as input. First, the function projects the four literal corner coordinates of the source image using the homography matrix and normalizes them by their homogeneous scale factor. Next, it combines these transformed cordinates with the original image bounds and computes the minimum and maximum spatial extents using floor and ceil operations. Finally, it applies safety clamps to prevent runaway memory allocation and returns the integer canvas boundaries (x_min, x_max, y_min, y_max) required to properly size the output warping grid.

Now that I have homography matrix and the size of the canvas for my final warped image I will create the final warped image by transforming the source image using the homography matrix and mapping it onto the destination image. I use two different methods and compare how each method affects the processing time and the final result of warping.

The warpImageNearestNeighbor function performs inverse warping taking the source image and homography matrix as input. First, it determines the output canvas dimensions and generates a grid of destination pixel coordinates, which it maps back to floating-point source coordinates using the inverse homography matrix and homogeneous division. Next, it rounds these source coordinates to the nearest integer pixel index and applies a validity mask to filter out-of-bounds locations. Finally, it samples the color values directly from the single nearest neighbor at the clamped indices across all channels, returning the warped image along with a binary alpha mask.

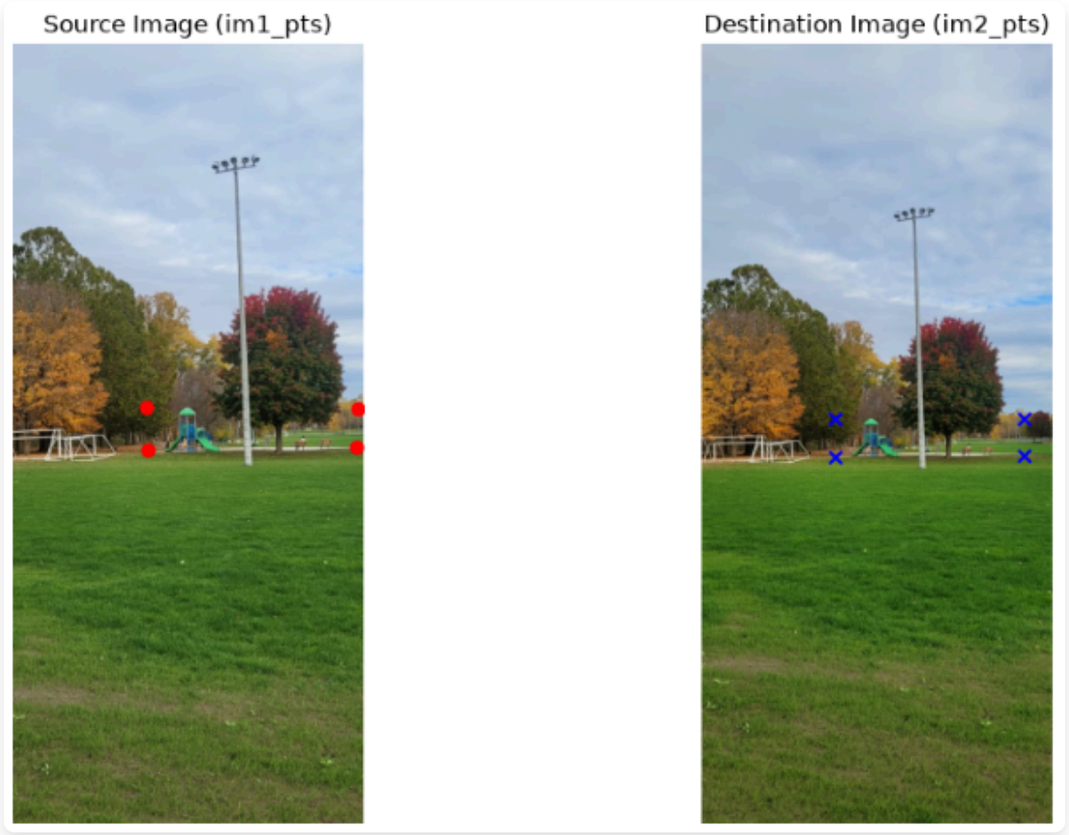
The warpImageBilinear function does the same inverse warping as the warpImageNearestNeighbor function but where it difers is how it samples the color values, insted of rounding to the closest pixel it takes a weighted average of four surrounding pixel neighbors across each color channel leading to a smooth boundary gradient instead of sharp boundaries avoiding jagged edges.

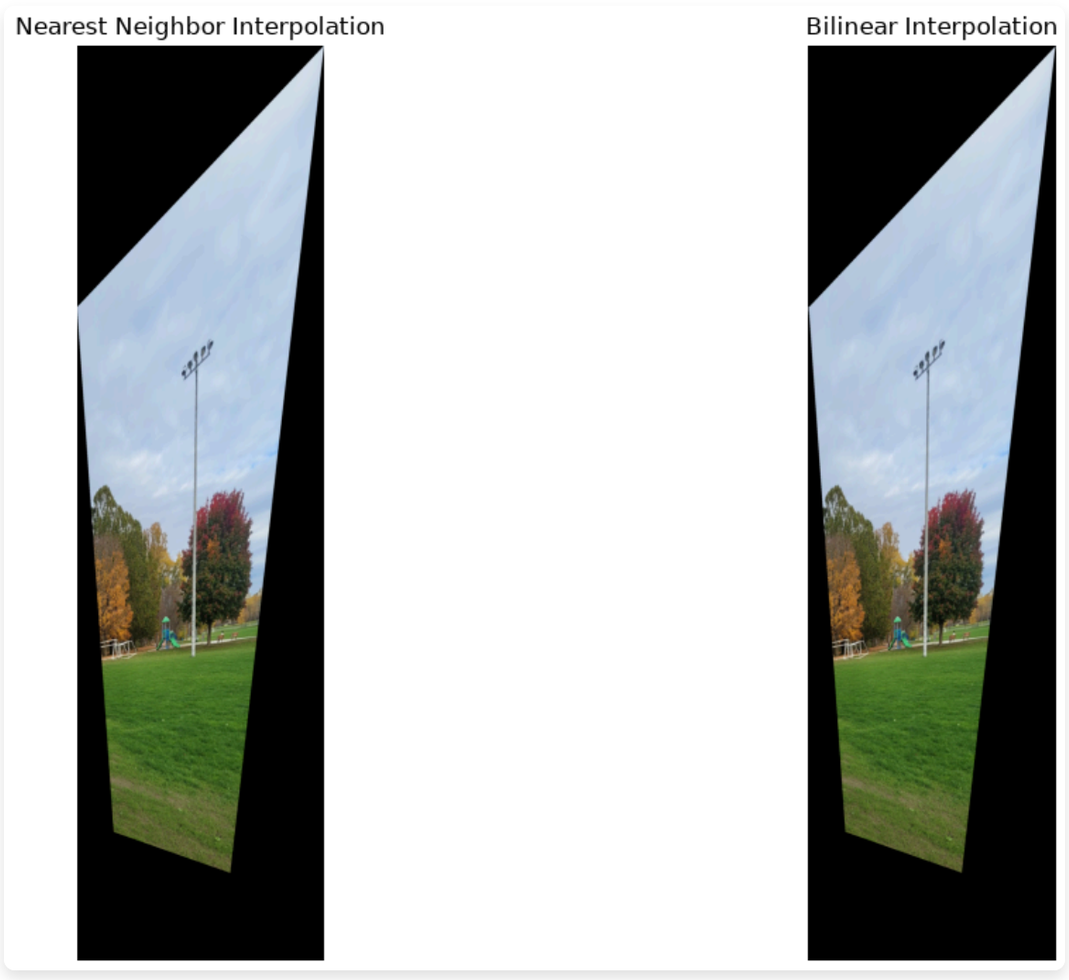


In the image above, I have selected the 4 corners of the source image and the target image, both in clockwise order to ensure that pixels from the source are corresponding to the same locations on the target image. In the bottom two

images, I have the two warped images from the NearestNeighbor and the Billienear functions shown.

The results of both the functions seem to have produced visibly identical results in this instance. The billinear function took a little longer to process the image than nearest neighbor function and the processing time would likely increase with the size and complexity of images.

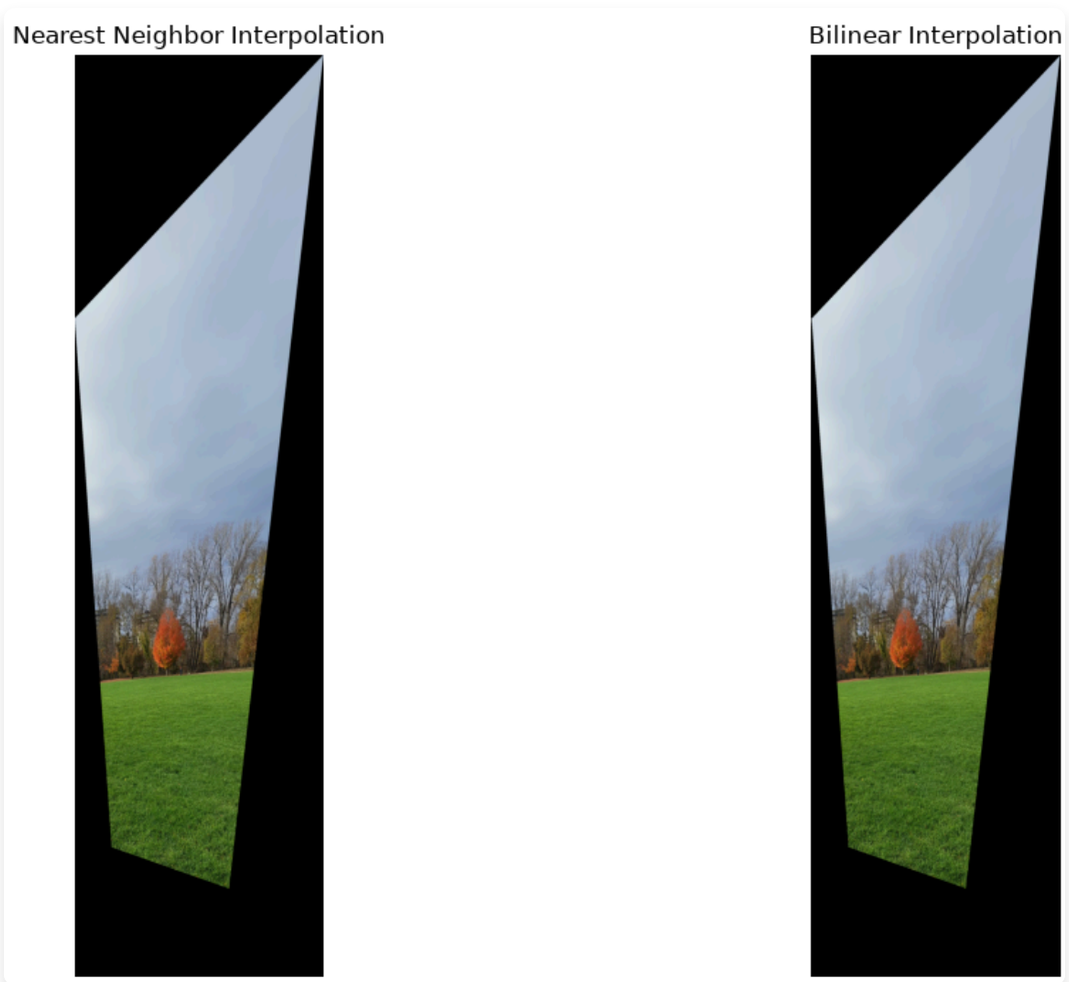




In this example, I took two photos of the playground from different distances and angles and warp them using both, bilinear and nearest neighbor, methods. For

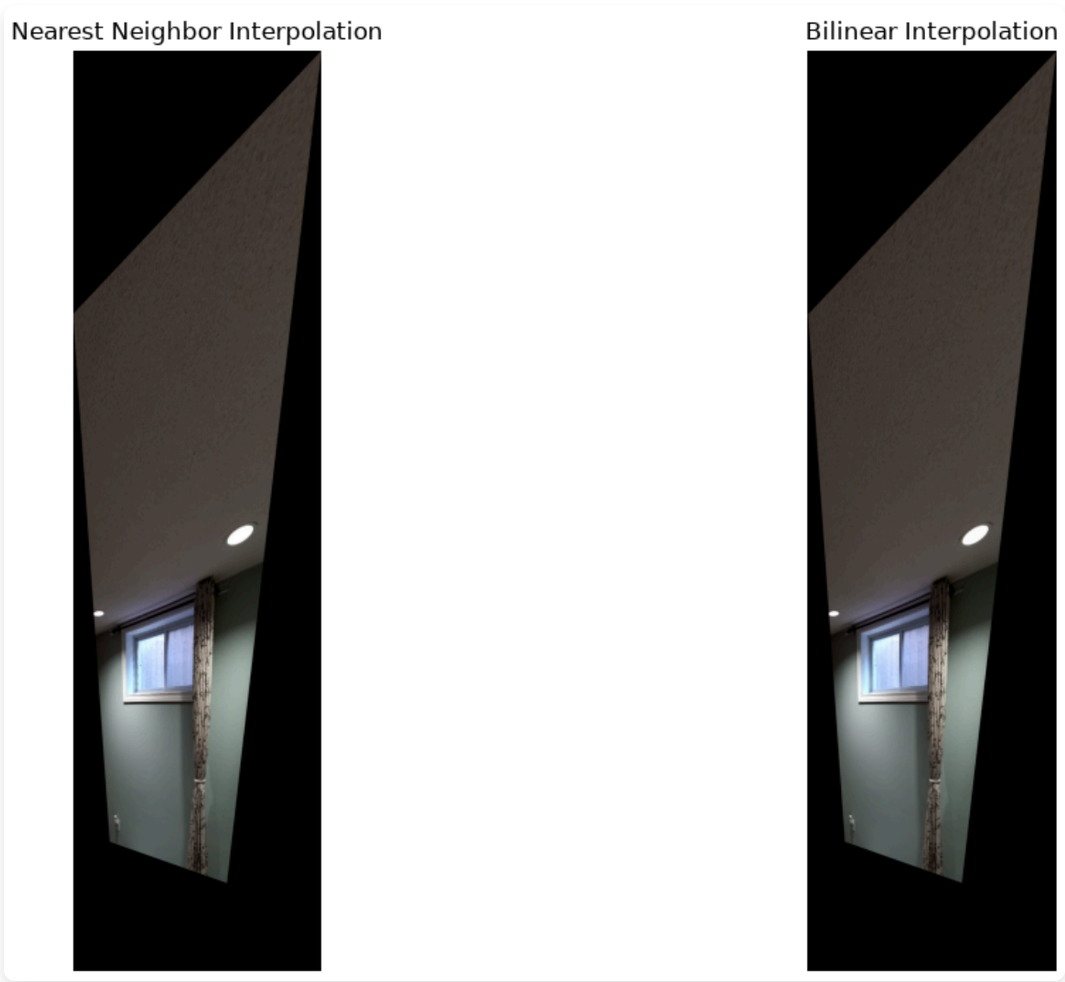
points of correspondances I choose the 4 points around the playground sandbox. The results for both the methods look almost idetical.





In this example, I have chosen a view with trees and buildings behind the tress. I took both images from different angles and for points of corresponace I chose the corners of one of the buildings. Again the results of both the interpolation methods look very similar.





The previous two photos didn't have any prominient feature to use to form a quadrilateral for corespondance so in this image I used a window and took images from the left and right of it. Again, the results of the inerpolations look identical.

```
# Rectification
import numpy as np
```

```
import cv2
import matplotlib.pyplot as plt

img1 = cv2.imread(r"C:\Users\Muhamm
img1 = cv2.cvtColor(img1, cv2.COLOR

pts1 = np.array([[244, 694],[621, 7

width, height = 400, 400
pts2 = np.array([
                [0, 0],
                [width, 0],
                [width, height],
                [0, height]
], dtype=np.float64)

H_recovered = computeH(pts1, pts2)

im_warped, alpha = warpImageBilinea

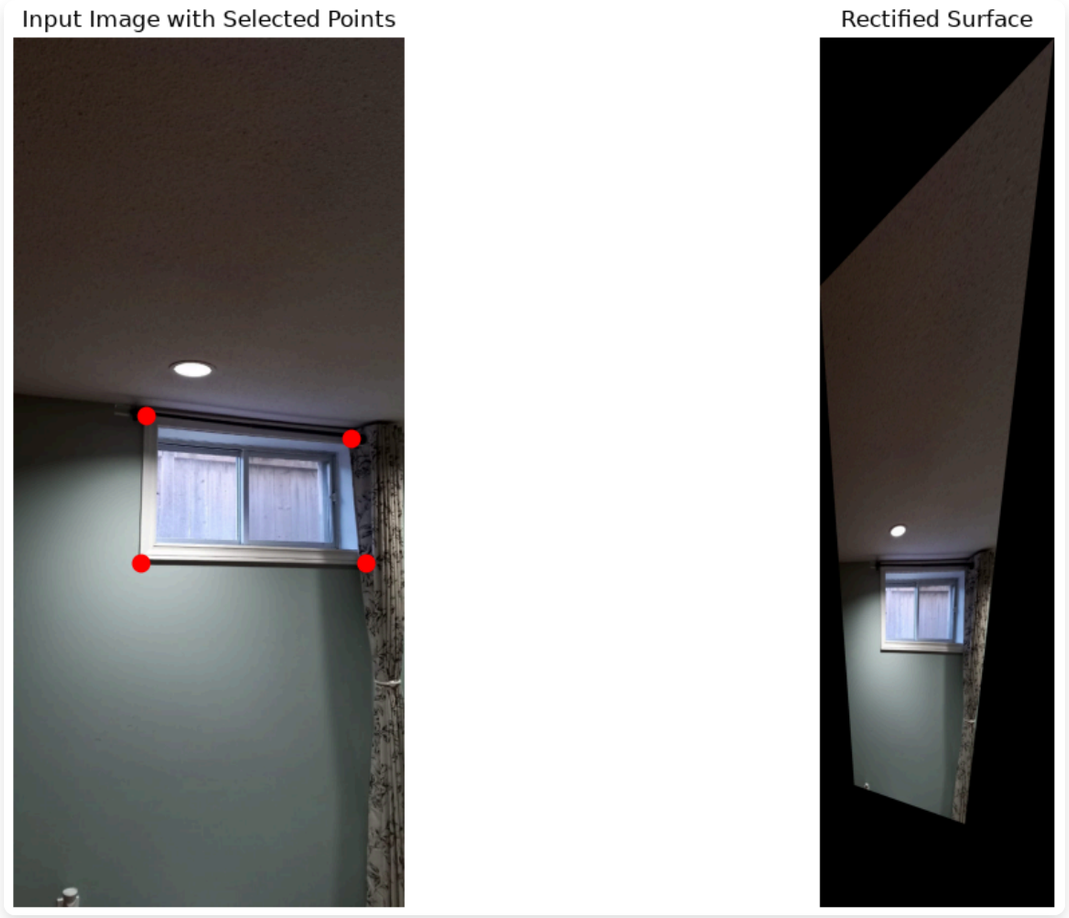
fig, axes = plt.subplots(1, 2, figs

axes[0].imshow(img1)
axes[0].scatter(pts1[:, 0], pts1[:,
axes[0].set_title("Input Image with
axes[0].axis('off')

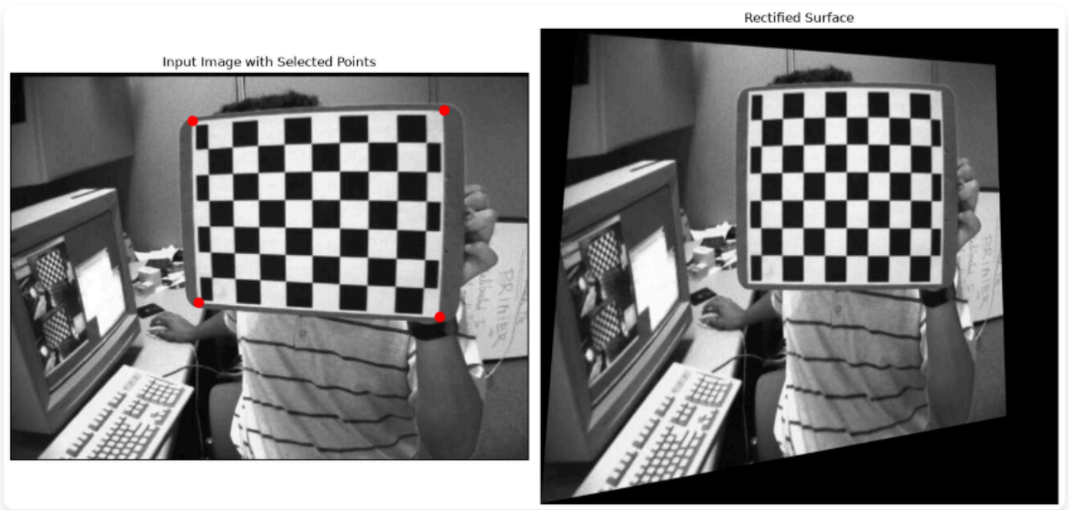
axes[1].imshow(im_warped)
axes[1].set_title("Rectified Surfac
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

To verify if my warping program works as expected so far, I ractified using a single image with a rectangular object of interest and ran it with our compute function to check if the image is warped to focus in on our object of interest and turn into a front facing image. The ractification takes manually input cordinates of the object of interest and then creates a flat canvas size based on the input cordinates, calclates the homography, performs inversion and warps the image to make object of inerest from slanted to flat front facing image. Below are some examples on actual images using the above code.



In the above image you can see that the window in the original image was captured with the camera positioned to the left of the window. our ractification function has correctly warpe the image so that it becomes flat and seems as if the camera was placed directly in front of it.



In the above image we revisit the chessboard.

Section 2: Image Blending and Mosaicing

Now that we have ractified that our program accurately warps the images, we now proceed to the next step. I will now stitch together multiple images to create one composite image. I use a set of 3 images in an urban setting and warp the images using the middle image as the reference image since it contains overlapping features from the left and right images. For each image, I choose different correspondance points that are common with midle reference image. The images I used to create the mosaic and the final mosaic is shown below. For correspondance points I used pixpy andmanually input the cordinates as arrays.





In the previous iteration I manually input the correspondance cordinates to create the mosaic. The warpImageBiliniearWithCanvas function calculates the weights of each image and uses distance transforms to generate a smooth alpha mask for the borders. I then implement the actual image blending directly in the main accumulation loop by transforming each image onto the global cordinate canvas, summing their weighted RGB values and mask them together. Finally, dividin the acumulated color grid by the total alpha weight normalizes the oerlapping regions, stitiching the three photographs together.

In this next iteration, I will implement automatic feature detection and correspondance point selection in the source and reference images to create a mosaic image. Below is the code to create automatic mosaic.

```
#Automated pipeline with correspondence visualization
import numpy as np
import cv2
import matplotlib.pyplot as plt

base = r"C:\Users\Muhammad Muneeb\Desktop\Documents\Masters\SYDE 671\Assignmen
img_a = cv2.cvtColor(cv2.imread(base + r"\a.jpg"), cv2.COLOR_BGR2RGB)
img_b = cv2.cvtColor(cv2.imread(base + r"\b.jpg"), cv2.COLOR_BGR2RGB)
img_c = cv2.cvtColor(cv2.imread(base + r"\c.jpg"), cv2.COLOR_BGR2RGB)

def extract_harris_features(img, max_corners=1000, threshold=0.01):
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY).astype(np.float32)
    corners = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)

    dst = cv2.dilate(corners, None)
    ret = dst > threshold * dst.max()
    y_idxs, x_idxs = np.where(ret)
    pts = np.stack((x_idxs, y_idxs), axis=-1)
```

```
    if len(pts) > max_corners:
        scores = dst[y_idxs, x_idxs]
        top_indices = np.argsort(scores)[::-1][:max_corners]
        pts = pts[top_indices]
    return pts

def extract_descriptors(img, pts):
    gray = cv2.cvtColor(img, cv2.COLOR_RGB2GRAY).astype(np.float32)
    h, w = gray.shape
    descriptors = []
    valid_pts = []

    for pt in pts:
        x, y = int(pt[0]), int(pt[1])

        if x - 20 < 0 or x + 20 >= w or y - 20 < 0 or y + 20 >= h:
            continue

        patch_40 = gray[y-20:y+20, x-20:x+20]
        patch_8 = cv2.resize(patch_40, (8, 8), interpolation=cv2.INTER_AREA)

        mean = np.mean(patch_8)
        std = np.std(patch_8)
        if std > 1e-5:
            norm_patch = (patch_8 - mean) / std
        else:
            norm_patch = patch_8 - mean

        descriptors.append(norm_patch.flatten())
        valid_pts.append([x, y])

    return np.array(valid_pts), np.array(descriptors)

def match_features_lowe(desc1, desc2, pts1, pts2, ratio_thresh=0.75):
    matches = []

    dist_matrix = np.linalg.norm(desc1[:, np.newaxis, :] - desc2[np.newaxis, :

    for i, row in enumerate(dist_matrix):
        sorted_idxs = np.argsort(row)
        if len(sorted_idxs) < 2:
            continue
        nn1, nn2 = sorted_idxs[0], sorted_idxs[1]
        if row[nn1] < ratio_thresh * row[nn2]:
            matches.append((pts1[i], pts2[nn1]))

    if len(matches) == 0:
        return np.empty((0, 2)), np.empty((0, 2))

    matches = np.array(matches)
    return matches[:, 0], matches[:, 1]

pts_a_raw = extract_harris_features(img_a)
pts_b_raw = extract_harris_features(img_b)
pts_a_pts, desc_a = extract_descriptors(img_a, pts_a_raw)
pts_b_pts1, desc_b1 = extract_descriptors(img_b, pts_b_raw)
```

```
pts_a_matched, pts_b_matched_for_a = match_features_lowe(desc_a, desc_b1, pts_

pts_c_raw = extract_harris_features(img_c)
pts_c_pts, desc_c = extract_descriptors(img_c, pts_c_raw)
pts_b_pts2, desc_b2 = extract_descriptors(img_b, pts_b_raw)
pts_c_matched, pts_b_matched_for_c = match_features_lowe(desc_c, desc_b2, pts_

fig, axes = plt.subplots(1, 3, figsize=(16, 6))

axes[0].imshow(img_a)
axes[0].scatter(pts_a_matched[:, 0], pts_a_matched[:, 1], color='red', s=40, m
axes[0].set_title("Image A Matches")
axes[0].axis('off')
axes[0].legend(loc='upper right')

axes[1].imshow(img_b)
axes[1].scatter(pts_b_matched_for_a[:, 0], pts_b_matched_for_a[:, 1], color='r
axes[1].scatter(pts_b_matched_for_c[:, 0], pts_b_matched_for_c[:, 1], color='c
axes[1].set_title("Image B (Reference) Matches")
axes[1].axis('off')
axes[1].legend(loc='upper right')

axes[2].imshow(img_c)
axes[2].scatter(pts_c_matched[:, 0], pts_c_matched[:, 1], color='cyan', s=40, l
axes[2].set_title("Image C Matches")
axes[2].axis('off')
axes[2].legend(loc='upper right')

plt.tight_layout()
plt.show()

def computeH(im1_pts, im2_pts):
    def norm_T(p):
        c = p.mean(axis=0)
        s = np.sqrt(2) / np.mean(np.linalg.norm(p - c, axis=1))
        return np.array([[s, 0, -s * c[0]],
                         [0, s, -s * c[1]],
                         [0, 0, 1]])

    T1, T2 = norm_T(im1_pts), norm_T(im2_pts)
    p1 = (T1 @ np.c_[im1_pts, np.ones(len(im1_pts))].T).T[:, :2]
    p2 = (T2 @ np.c_[im2_pts, np.ones(len(im2_pts))].T).T[:, :2]

    A = []
    for (x, y), (xp, yp) in zip(p1, p2):
        A.append([-x, -y, -1, 0, 0, 0, x * xp, y * xp, xp])
        A.append([0, 0, 0, -x, -y, -1, x * yp, y * yp, yp])
    _, _, Vt = np.linalg.svd(np.array(A, dtype=np.float64))
    Hn = Vt[-1].reshape(3, 3)

    H = np.linalg.inv(T2) @ Hn @ T1
    return H / H[2, 2]

def ransac_homography(pts1, pts2, num_iterations=1500, threshold=4.0):
    num_pts = len(pts1)
    if num_pts < 4:
```

```
        raise ValueError("Not enough matching points found to run RANSAC (need

    best_H = None
    max_inliers = -1
    pts1_h = np.hstack([pts1, np.ones((num_pts, 1))])

    for _ in range(num_iterations):
        idx = np.random.choice(num_pts, 4, replace=False)
        H_sample = computeH(pts1[idx], pts2[idx])

        projected = (H_sample @ pts1_h.T).T
        eps = 1e-8
        projected[:, :2] /= (projected[:, 2:3] + eps)

        errors = np.linalg.norm(projected[:, :2] - pts2, axis=1)
        inlier_mask = errors < threshold
        num_inliers = np.sum(inlier_mask)

        if num_inliers > max_inliers:
            max_inliers = num_inliers
            best_H = H_sample

    if max_inliers >= 4:
        projected = (best_H @ pts1_h.T).T
        projected[:, :2] /= (projected[:, 2:3] + 1e-8)
        final_mask = np.linalg.norm(projected[:, :2] - pts2, axis=1) < thresho
        if np.sum(final_mask) >= 4:
            best_H = computeH(pts1[final_mask], pts2[final_mask])

    print(f"RANSAC finished with {max_inliers} inliers out of {num_pts} matche
    return best_H

def check_H(name, H, src, dst, img):
    pred = H @ np.c_[src, np.ones(len(src))].T
    pred = pred[:2] / pred[2]
    err = np.linalg.norm(pred - dst.T, axis=0)
    h, w = img.shape[:2]
    corners = np.array([[0, 0, 1], [w - 1, 0, 1], [w - 1, h - 1, 1], [0, h - 1
    w_vals = (H @ corners)[2]
    print(f"{name}: reproj err (px) = {np.round(err, 2)}")
    print(f"{name}: w at corners   = {np.round(w_vals, 3)}  (should all be > 0
    return np.all(w_vals > 0.05)

H_a_to_b = ransac_homography(pts_a_matched, pts_b_matched_for_a)
H_c_to_b = ransac_homography(pts_c_matched, pts_b_matched_for_c)
H_b_to_b = np.eye(3)

ok_a = check_H("H_a_to_b", H_a_to_b, pts_a_matched[:4], pts_b_matched_for_a[:4
ok_c = check_H("H_c_to_b", H_c_to_b, pts_c_matched[:4], pts_b_matched_for_c[:4

images = [img_a, img_b, img_c]
homographies = [H_a_to_b, H_b_to_b, H_c_to_b]

all_x, all_y = [], []
for i, (img, H) in enumerate(zip(images, homographies)):
    h, w = img.shape[:2]
```

```
    corners = np.array([[0, 0, 1], [w - 1, 0, 1], [w - 1, h - 1, 1], [0, h - 1
    t = H @ corners
    t = t / t[2, :]
    print(f"Image {i} transformed corners X: {np.round(t[0], 1)}  Y: {np.round
    all_x.extend(t[0])
    all_y.extend(t[1])

x_min, x_max = int(np.floor(min(all_x))), int(np.ceil(max(all_x)))
y_min, y_max = int(np.floor(min(all_y))), int(np.ceil(max(all_y)))
out_w = x_max - x_min + 1
out_h = y_max - y_min + 1
print(f"Canvas: {out_w} x {out_h}  (x {x_min}..{x_max}, y {y_min}..{y_max})")

if not (ok_a and ok_c) or out_w > 4000 or out_h > 4000:
    raise SystemExit(
        "Homography looks degenerate (huge canvas or w <= 0 at a corner). "
        "Check feature matching performance or adjust RANSAC threshold."
    )

T = np.array([[1, 0, -x_min],
              [0, 1, -y_min],
              [0, 0, 1]], dtype=np.float64)

def warpImageBilinearWithCanvas(im, H_global, out_w, out_h):
    h, w = im.shape[:2]

    mask = np.ones((h, w), dtype=np.uint8) * 255
    mask[0, :] = 0; mask[-1, :] = 0; mask[:, 0] = 0; mask[:, -1] = 0
    dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
    alpha_src = dist / (dist.max() + 1e-5)

    X_out, Y_out = np.meshgrid(np.arange(out_w), np.arange(out_h))
    coords_out = np.vstack((X_out.ravel(), Y_out.ravel(), np.ones(out_w * out_

    H_inv = np.linalg.inv(H_global)
    coords_src = H_inv @ coords_out
    wz = coords_src[2, :]
    wz[np.abs(wz) < 1e-12] = 1e-12
    coords_src = coords_src / wz

    X_src = coords_src[0, :].reshape(out_h, out_w)
    Y_src = coords_src[1, :].reshape(out_h, out_w)

    x0 = np.floor(X_src).astype(np.int64); x1 = x0 + 1
    y0 = np.floor(Y_src).astype(np.int64); y1 = y0 + 1

    wa = (x1 - X_src) * (y1 - Y_src)
    wb = (X_src - x0) * (y1 - Y_src)
    wc = (x1 - X_src) * (Y_src - y0)
    wd = (X_src - x0) * (Y_src - y0)

    valid = (x0 >= 0) & (x1 < w) & (y0 >= 0) & (y1 < h)

    x0c = np.clip(x0, 0, w - 1); x1c = np.clip(x1, 0, w - 1)
    y0c = np.clip(y0, 0, h - 1); y1c = np.clip(y1, 0, h - 1)
```

```
    warped = np.zeros((out_h, out_w, 3), dtype=np.float64)
    for c in range(3):
        ch = im[:, :, c].astype(np.float64)
        val = (wa * ch[y0c, x0c] + wb * ch[y0c, x1c] +
               wc * ch[y1c, x0c] + wd * ch[y1c, x1c])
        val[~valid] = 0
        warped[:, :, c] = val

    alpha = (wa * alpha_src[y0c, x0c] + wb * alpha_src[y0c, x1c] +
             wc * alpha_src[y1c, x0c] + wd * alpha_src[y1c, x1c])
    alpha[~valid] = 0
    return warped, alpha

acc_rgb = np.zeros((out_h, out_w, 3), dtype=np.float64)
acc_alpha = np.zeros((out_h, out_w), dtype=np.float64)

for img, H in zip(images, homographies):
    warped, alpha = warpImageBilinearWithCanvas(img, T @ H, out_w, out_h)
    acc_rgb += warped * alpha[:, :, np.newaxis]
    acc_alpha += alpha

acc_alpha[acc_alpha == 0] = 1.0
mosaic = np.clip(acc_rgb / acc_alpha[:, :, np.newaxis], 0, 255).astype(np.uint

plt.figure(figsize=(12, 6))
plt.imshow(mosaic)
plt.title("Automated RANSAC Stitched Image Mosaic")
plt.axis("off")
plt.show()
```





The code automatically picks correspondance points in the input images and matches them to similar points on the reference image. Firstly, Harris corner detection is used to detect prominent interest points by converting the images to greyscale and identifiying regions where the gradient changes sharply in more than one direction.

For each of these corners, a 40x40 pixel neighborhood is extracted, resized to a 8x8 matrix and normalized to ensure invariance to lighting and contrast changes.

Next, the features are matched between images by calculating the eucleadean distance between the normalized matrices. For each point two nearest neighbors are picked and tested with Lowe's ratio test, rejecting ambigous matches where multiple targets look similar.

Automated matching inevitably yields incorrect matches. The RANSAC (Random Sample Consensus) samples random subsets of 4 point correspondences, computes a trial homography matrix using, and counts how many total matching points agree with that model within a pixel error threshold. The model with the highest number of consistent inliers is selected as the optimal homography.

Comparing the results of the manual mosaic and the automated mosaic it is clear that manual mosaic produced better results since I picked points at distinct sharp corners where there were antural boundaries, such as roof and trees or the edge of a building and also spaced the correspondance points further apart from each other to ensure sufficient overlap between images.

Harris corner detector concentrated matches on a very small, localized cluster (mostly clustered on the distant apartment building facade shown in the top match visualizations), leaving large portions of the image unconstrained. Simple normalized intensity patches combined with Lowe's ratio test are sensitive to repetitive textures (like windows on high-rise apartment blocks). This causes incorrect cross-matches that slip past the RANSAC threshold if the initial point pool lacks good geometric spread. When point correspondences are clustered in a small region rather than spread across the image plane, the resulting homography matrix becomes ill-conditioned. Small pixel noise leads to massive angular distortions when projected across the wider canvas

In order to improve the results for the automated pipeline, instead of the Harris corner detection we can use other corner detection methods that space out the correspondance points evenly throughout the image. Lowering the RANSAC reprojection error threshold can further help improve the results.